PRACTICAL 7

Aim :

Implement JWT-based authentication and server-side input validation in the Express middleware pipeline. Users must be able to register and log in securely, passwords must be hashed with bcrypt, and task routes must reject requests without a valid JWT.

Code :

backend/models/User.js :-

```
const mongoose = require('mongoose');



const userSchema = new mongoose.Schema({

  name: { type: String, required: true, trim: true },

  email: {

    type: String,

    required: true,

    unique: true,

    lowercase: true,

    trim: true,

    match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Enter a valid email address']

  },

  password: { type: String, required: true, minlength: 6 }

}, { timestamps: true });



module.exports = mongoose.model('User', userSchema);
```

backend/middleware/auth.js :-

```
const jwt = require('jsonwebtoken');
```

```
function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Authorization token required' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}

module.exports = requireAuth;
```

Code :

backend/routes/auth.js :-

```
router.post('/register', asyncHandler(async (req, res) => {
  const { name, email, password } = req.body;
  const existingUser = await User.findOne({ email: email?.toLowerCase().trim() });
  if (existingUser) return res.status(409).json({ error: 'Email is already registered' });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hashedPassword });
  res.status(201).json({ user, token: createToken(user) });
}));

router.post('/login', asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email: email?.toLowerCase().trim() });
  const matches = user && await bcrypt.compare(password || '', user.password);
  if (!matches) return res.status(401).json({ error: 'Invalid email or password' });
  res.status(200).json({ user, token: createToken(user) });
```

```
}));
```

backend/routes/tasks.js :-

```
const requireAuth = require('../middleware/auth');
const router = express.Router();
router.use(requireAuth);
router.get('/', asyncHandler(async (req, res) => {
  const tasks = await Task.find().sort({ createdAt: -1 });
  res.status(200).json(tasks);}));
```

Protected request header :-

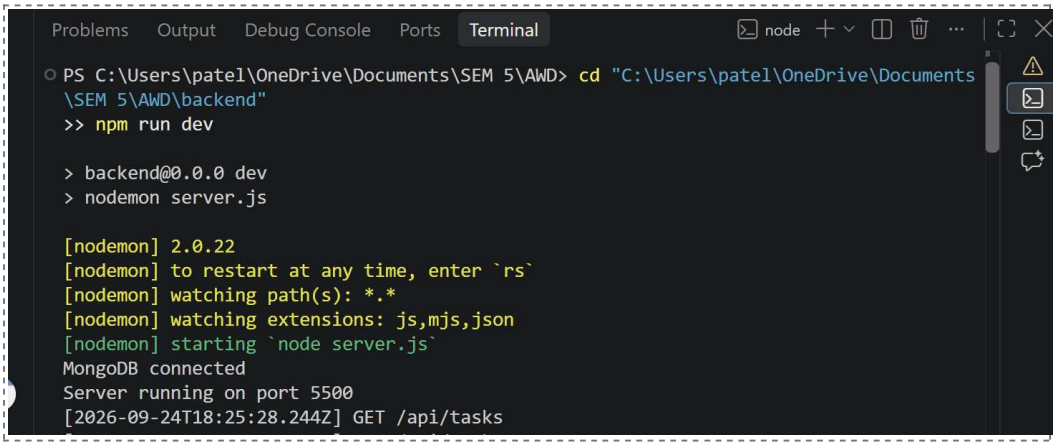
Authorization: Bearer <JWT_TOKEN>

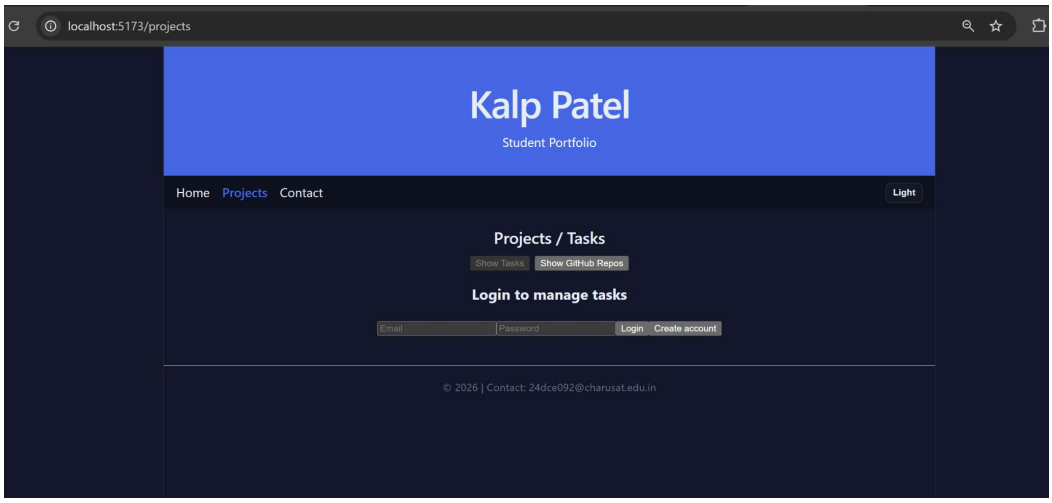
Testing :

Start the backend and frontend in separate terminals. Use Postman or the React interface to test registration, login, and protected task requests.

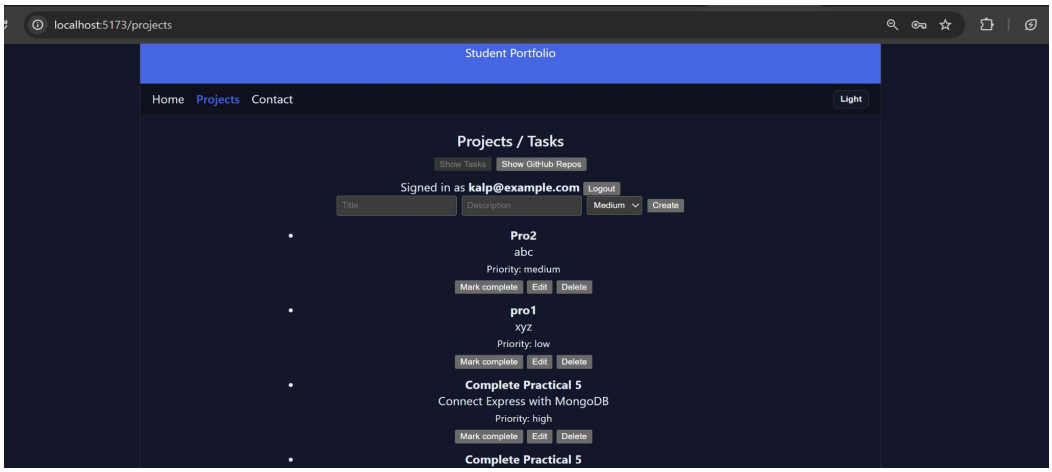
cd backend
npm run dev
cd frontend
npm run dev

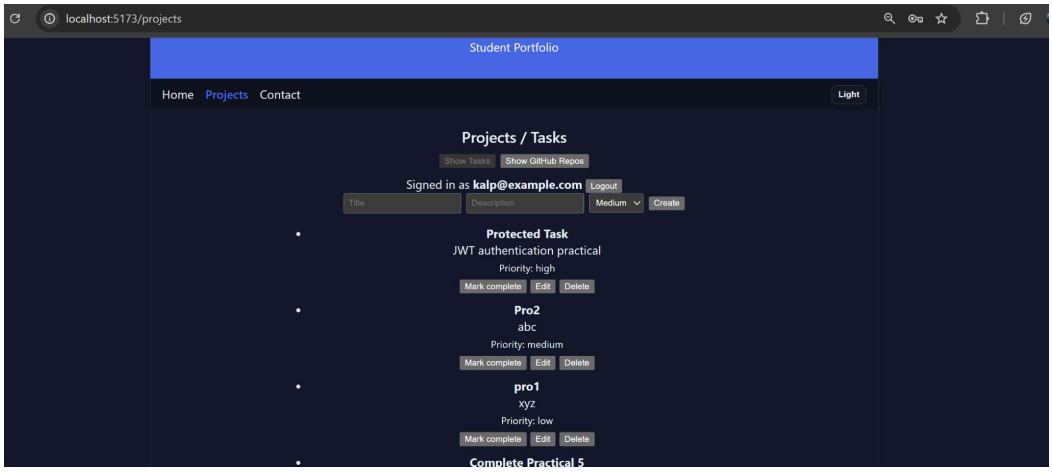
Output :

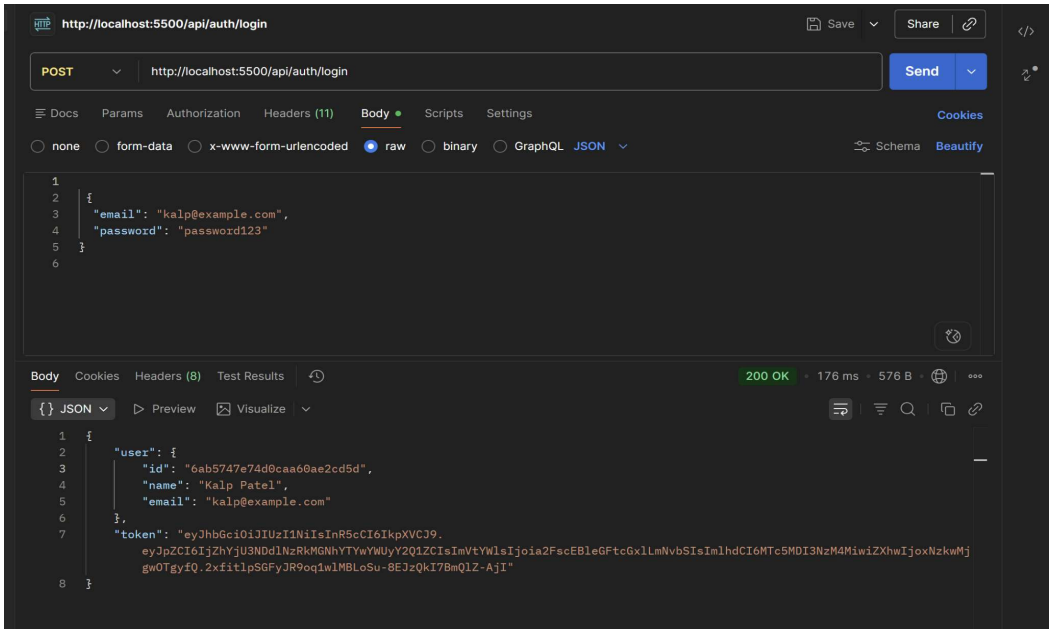




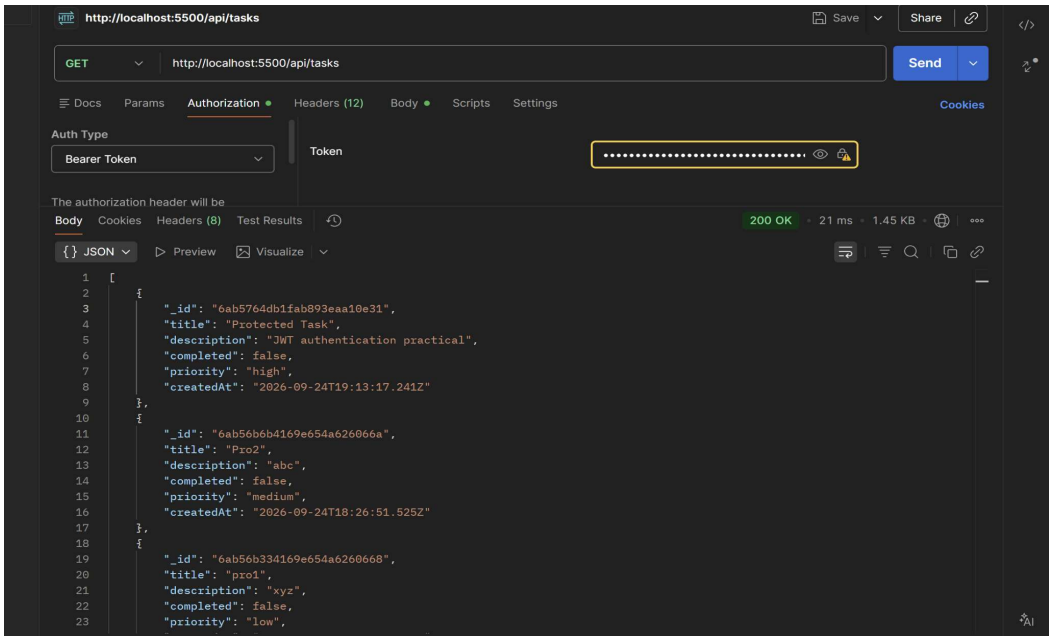
Output :

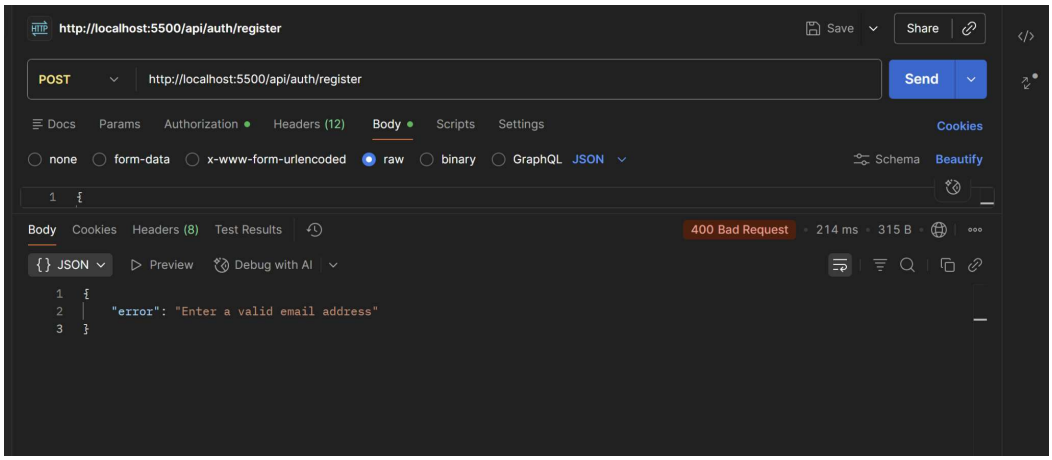

localhost:5173/projects
Student Portfolio
Home Projects Contact
Light
Projects / Tasks
Show Tasks
Show GitHub Repos
Signed in as kalp@example.com
Logout
Title
Description
Medium
Create
Pro2
abc
Priority: medium
Mark complete
Edit
Delete
pro1
xyz
Priority: low
Mark complete
Edit
Delete
Complete Practical 5
Connect Express with MongoDB
Priority: high
Mark complete
Edit
Delete
Complete Practical 5


localhost:5173/projects
Student Portfolio
Home Projects Contact
Light
Projects / Tasks
Show Tasks
Show GitHub Repos
Signed in as kalp@example.com
Logout
Title
Description
Medium
Create
Protected Task
JWT authentication practical
Priority: high
Mark complete
Edit
Delete
Pro2
abc
Priority: medium
Mark complete
Edit
Delete
pro1
xyz
Priority: low
Mark complete
Edit
Delete
Complete Practical 5


http://localhost:5500/api/auth/login
Save
Share
POST
http://localhost:5500/api/auth/login
Send
Docs Params Authorization Headers (11) Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "kalp@example.com",
"password": "password123"
}
Body Cookies Headers (8) Test Results
200 OK 176 ms 576 B
{ } JSON Preview Visualize
{
"user": {
"id": "6ab5747e74d0caa60ae2cd5d",
"name": "Kalp Patel",
"email": "kalp@example.com"
},
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhYjU3NDdlNzRkMGNhYTYwYWUyY2Q1ZCIsImVtYWlsIjoia2FscEBleGFtcGxlLmNvbSIsImlhdCI6MTc5MDI3NzM4MiwiZXhwIjoxNzkwMj
gwOTgyfQ.2xfitlpSGFyJR9oq1wlMBLoSu-8EJzQkI7BmQlZ-AjI"
}

Output :





Result :

JWT authentication was successfully added to the Task Management application. Passwords are hashed with bcrypt before storage, login returns a signed token with a one-hour expiry, and all task routes reject missing or invalid tokens. Server-side validation returns structured JSON errors for malformed input.